National College of Ireland Project

Submission Sheet

Student Name: **Rana Hasnat Ahmed**

Student ID: x24320170

Programme: MSc in Artificial Intelligence **Year:** 2025/2026

Module: Engineering and Evaluating Artificial Intelligence (MSCAI1)

Lecturer: Dr Abdul Razzaq

Submission Due Date: 26/03/26

Project Title: Modular Email Classification Architecture: Chained Multi-Output Classification and Hierarchical Modelling

GitHub Repository: EEA_MultiLabel_Email_Classifier - **https://github.com/hasnat360/EEA_MultiLabel_Email_Classifier**

Word Count:

Academic Integrity Certificate Verification Code: pevwKcF2Z6

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement - Starting **Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: https://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature: Rana Hasnat Ahmed

Date: March 23, 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Engineering and Evaluating Artificial Intelligence]

[Modular Email Classification Architecture: Chained Multi-Output Classification and Hierarchical Modelling]

Your Name/Student Number	Course	Date
Rana Hasnat Ahmed - X24320170	**Engineering and Evaluating Artificial Intelligence**	March 23, 2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
ChatGPT (OpenAI GPT-4/5)	Used for reviewing report structure,improving academic wording, checking compliance with coursework requirements, and verifying ML model explanations. The ML model implementation, coding implementation, and experimental analysis were independently developed by the student.	https://chat.openai.com

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

ChatGPT (OpenAI GPT-4/5)	
ChatGPT was used as a support tool for reviewing the structure of the coursework report, checking formatting consistency, and improving clarity of explanations related to machine learning concepts. The mathematical formulation, AMPL model implementation, dataset preprocessing, sensitivity analysis, and interpretation of results were developed independently by the student.	
"Review this CA1 proposal and check whether it follows the NCI template structure and Harvard referencing format."	The AI provided structured feedback identifying minor citation punctuation errors, reference ordering adjustments, and formatting improvements required for compliance with Harvard style and the CA1 template.

Evidence of AI Usage

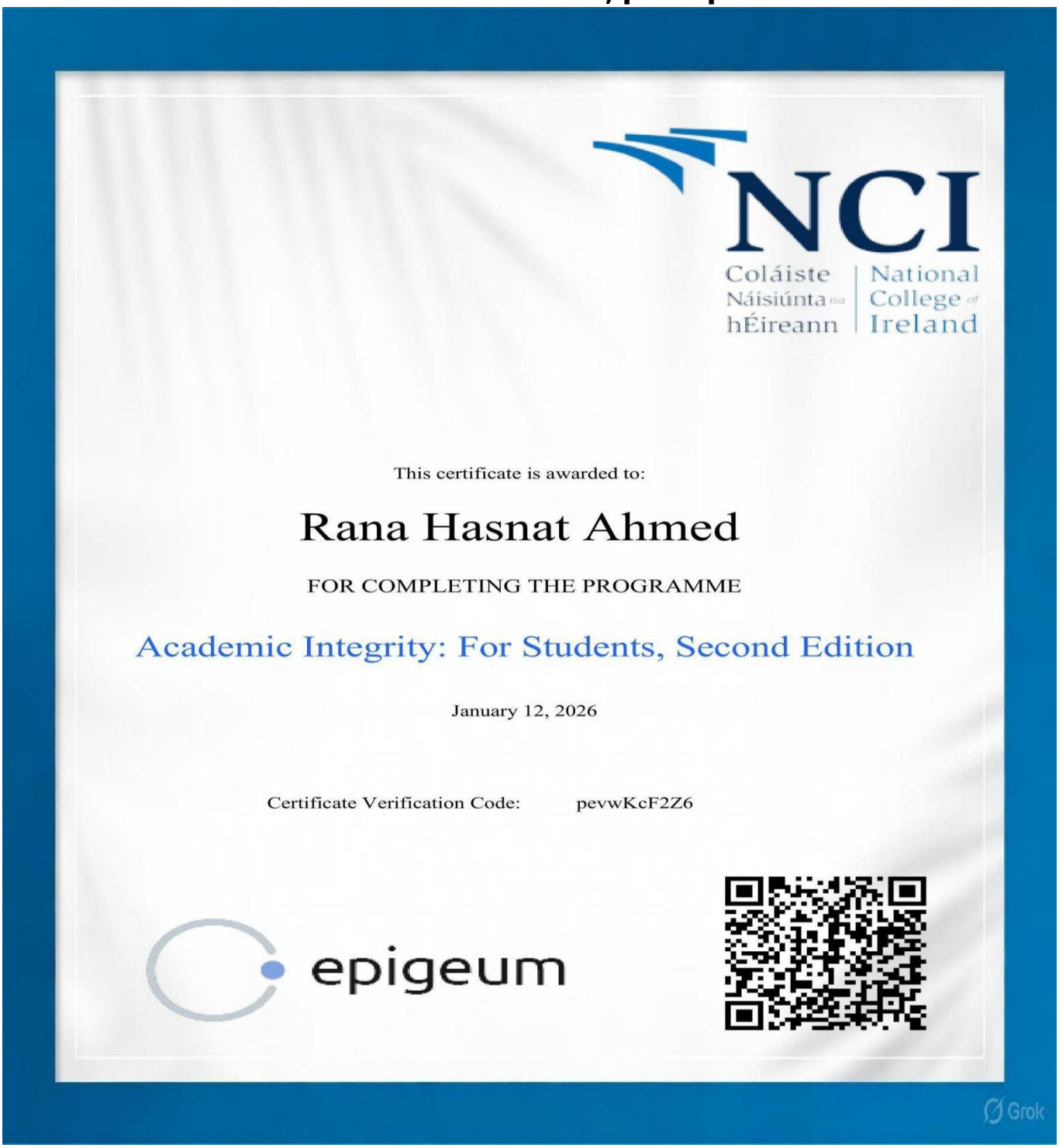
This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

Additional Evidence:

Academic Integrity Acknowledgement Supplement

The arrow below indicates certificate number, please put this in the above field



NCI
Coláiste Náisiúnta na hÉireann | National College of Ireland

This certificate is awarded to:

Rana Hasnat Ahmed

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 12, 2026

Certificate Verification Code: pevwKcF2Z6

epigeum

Grok

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

1. CA Requirement Summary

The brevity dictates that the first software-engineering property to be maintained is the separation of concerns between preprocessing and modelling, the second property is the consistent input encapsulation so that models can be fed the same train/test structure, and the final property is the abstraction via a reusable model interface.

The brief requests that there be two other design options concerning the classification of Type 2, Type 3 and Type 4. Design Choice 1 should consider the successively deeper combinations of labels as chained outputs whereas Design Choice 2 should behave by routing each instance to the subsequent models based on the prediction of the previous level. The report should display the drawing of every design option and give the tables of the components, connectors and data elements.

The final architectures do not use Type 1 as a target prediction, as the brief clearly permits the emphasis to be on Type 2, Type 3, and Type 4.

CA need	Code evidence	Status	Comment
Separation of preprocessing and modelling	preprocess.py, embeddings.py, modelling.py, main.py	Met	Cleaning, vectorization, and training are split across dedicated modules.
Input encapsulation	data_model.py -> Data / Filtered Data	Met	Train/test arrays and labels are wrapped before reaching the model.
Abstraction	base.py + randomforest.py	Met	The concrete Random Forest model follows the abstract interface.
Design Choice 1 implementation	create_chained_cols() + chained_model_predict()	Met	Combined targets are formed and evaluated chain by chain.
Design Choice 2 implementation	hierarchical_model_predict()	Met	Previous predictions route samples to branch-specific models.
Design sketches + architecture report	Included in this rebuilt report	Met	Figures and mapping tables are embedded below.

2. Verified Implementation Architecture

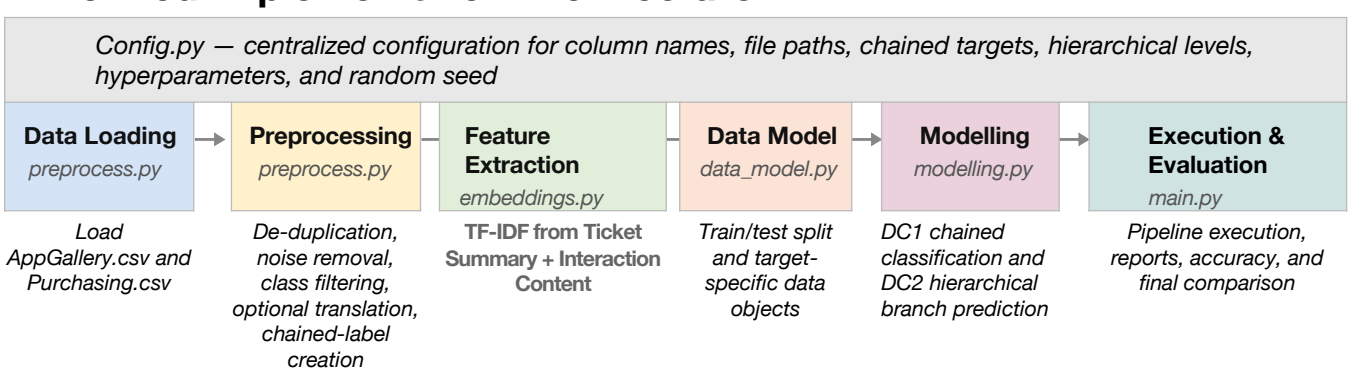


Figure 1. Modular and object-oriented implementation architecture of the submitted email classification system.

The codebase is structured and has a well defined responsibility divide. The configuration is stored in config.py and data loaded and preprocessed in preprocess.py, TF-IDF features of the ticket summary and interaction content are created in embeddings.py, the target-specific train/test data objects are prepared in data-model.py, the actual classifier implementation is found in randomforest.py, and coordination of the two design choices and their performance is facilitated in modelling.py and main.py. This architecture is suitable to the CA as it provides support to maintainability and future extensions. With the same abstract interface, the possibility of a new model is possible without having to recode the preprocessing or data preparation logic.

File	Architectural role	Why it matters
Config.py	Central configuration	Keeps labels, thresholds, seed, vectorizer settings, and file paths in one place.
preprocess.py	Input preparation	Loads both CSV files, standardizes labels, removes duplicates, and cleans message noise.
embeddings.py	Feature construction	Builds one shared TF-IDF feature space from ticket summary and interaction content.
data_model.py	Input encapsulation	Creates Data and Filtered Data objects containing X_train, X_test, y_train, and y_test.
base.py	Abstraction layer	Enforce train, predict, and print_results methods for any future model.
randomforest.py	Concrete implementation	Implements the actual Random Forest classifier used in the final run.
modelling.py	Design-choice controller	Runs chained modelling and hierarchical routing logic.
main.py	Execution entry point	Orchestrates preprocessing, feature creation, both architectures, and final comparison output.

3. Data Preparation and Feature Construction

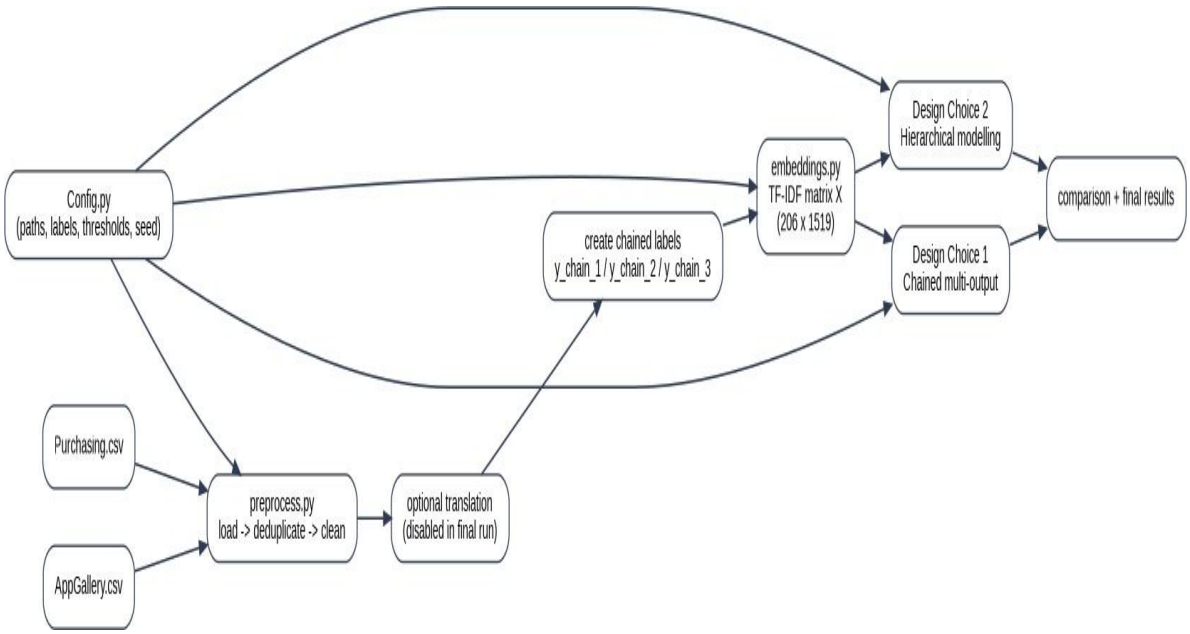


Figure 2. End-to-end pipeline from raw CSV files to both design choices.

The project is read by the AppGallery.csv and the Purchasing.csv, Type 1 columns are renamed into y1 to y4, the rows with no names dropped, and only columns with valid Type 2 labels are retained. Duplicate pieces of tickets are eliminated by ticket id such that emailed email chains do not distort the text representation. A noise-elimination step then deglomerates reply to lexical artefacts (markers, dates, support signatures, URLs, emails, and so on).

The result of the last cleaning step is a TF-IDF matrix created out of joint ticket summary and interaction content. This maintains separation of concerns since the modelling layer only takes numeric features and does not even require knowledge of the clean-up logic in the text.

There is a preprocessing option translation but it is disabled during the final verified execution. This can be accepted since translation is not a requirement of the brief and omission of it increases reproducibility.

Element	Observed value	Evidence from run	Interpretation
Records after preprocessing	206	Printed by main.py	All later modelling is based on the cleaned dataset.
Valid y2 labels	206 / 206	Printed by main.py	Type 2 is available for every retained record.
Valid y3 labels	171 / 206	Printed by main.py	Some deeper labels are missing, so filtered subsets are needed.
Valid y4 labels	163 / 206	Printed by main.py	The deepest level uses the smallest valid subset.
TF-IDF matrix shape	(206, 1519)	Printed by main.py	A shared feature space is used by both architectures.

4. Design Choice 1 - Chained Multi-Output Classification

Design Choice 1 is one that is not changing the feature flow. Rather, it varies the definition of the targets in every step. The identical cleaned data and identical global TF-IDF occurrence are reused, however the label is increasingly more profound: y2, y2 | y3, then y2 | y3 | y4. This is the reliance of subsequent labels on previous labels but the architectural flow is maintained simple.

Full Working Example for Design Choice 1

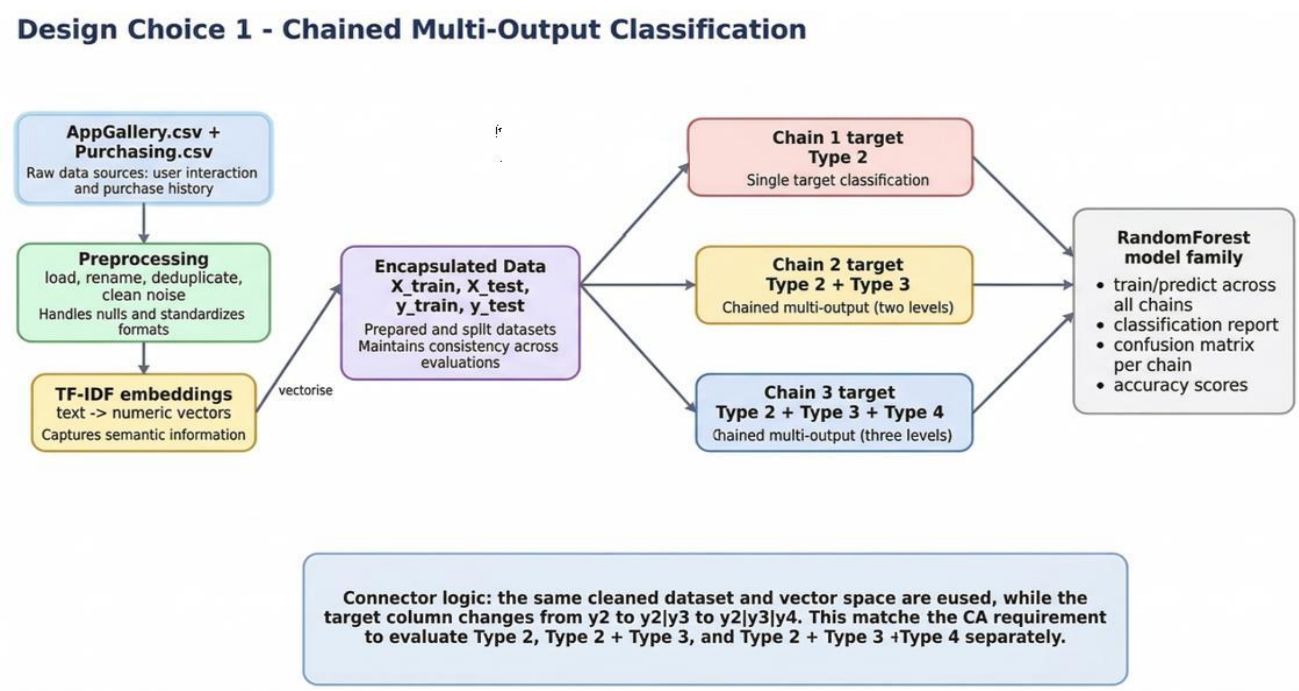


Figure 3. Sketch of Design Choice 1 with progressively combined targets.

Components	Connectors	Data elements
Config.py; preprocess.py; embeddings.py; data_model.py; base.py;randomforest.py; modelling.py	Cleaned dataframe -> create_chained_cols() -> TF-IDF matrix X -> Data object -> RandomForest train/predict -> report metrics	cleaned dataframe; y_chain_1; y_chain_2; y_chain_3; X_train/X_test; y_train/y_test; confusion matrices

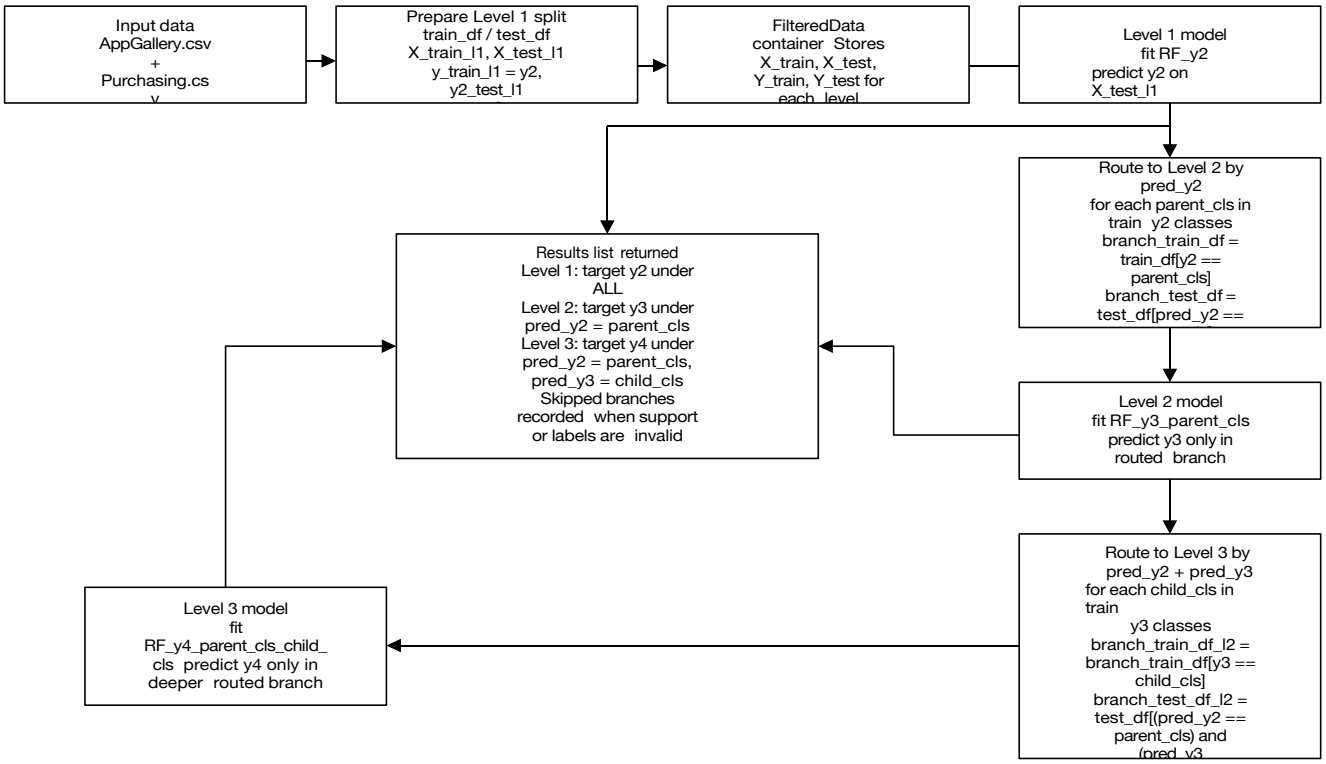
The main point in architecture is that the sequence of connectors is global and linear. The target column varies at the level of chains. This is why Design Choice 1 can be implemented and explained easily, as well as compared because all three models are based on a single overall pipeline.

Chain	Target	Valid rows	Classes	Accuracy
chain_1	y2	206	3	0.7619
chain_2	y2 | y3	171	11	0.6061
chain_3	y2 | y3 | y4	163	12	0.7931

Interpretation: chain 1 provides baseline Type 2 classification performance, chain 2 is a harder task because the label space grows significantly when Type 3 is included in the target. chain 3 is more accurate in the last run, but it cannot be interpreted as a strict increase in challenge since it is tested over a much smaller filtered subset with a different class distribution.

5. Design Choice 2 - Hierarchical Modelling

Design choice 2 alters the architecture. The first model is a prediction of y2 using the complete data. It is the expected y2 and this is used to direct every test sample to a branch-specific model of y3. Another subsequent forecasted pair (pred_y2, pred_y3) has the capability to re-feed the sample to a branch-specific y4 model. The design is more indicative of dependency and error propagation through the hierarchy.



Sketch of Design Choice 2 showing hierarchical branch-specific prediction paths.

Components	Connectors	Data elements
Same core modules, with modelling.py acting as a branch-aware controller that creates multiple RandomForest instances.	y2 prediction -> route by pred_y2 -> branch y3 models -> route by (pred_y2, pred_y3) -> branch y4 models or skip.	full-data y2 split; routed subsets; branch train/test arrays; parent predictions; low-support skip notes.

This architecture is more rich than Design Choice 1 in the sense that the model execution is based on past projections. It is hence more aligned to the spirit of hierarchical decision making but at the same time prone to propagation of errors and scarce data in the lower branches.

Level	Parent route	Target	Result	Comment
L1	ALL	y2	0.7619	Top-level classifier on the full dataset.
L2	pred_y2 = Others	y3	Skipped	No valid training labels for y3.
L2	pred_y2 = Problem/Fault	y3	0.7647	Best performing level-2 branch.
L3	Problem/Fault + Coupon/Gifts/Points Issues	y4	0.6667	Very small routed subset (6 test cases).
L3	Problem/Fault + Gallery-Install/Upgrade	y4	0.4286	Very small routed subset (7 test cases).
L2	pred_y2 = Suggestion	y3	0.6667	Stable branch with meaningful support.
L3	Suggestion + Payment	y4	0.6429	Best supported level-3 branch in Suggestion path.
L3	Other low-support branches	y4	Skipped	Train subset too small or insufficient support after filtering.

Interpretation: the skips in the code are not code failures. They are strategic measures to prevent testing of unreliable models on small subsets. The 1.0000 scores are also to be interpreted with care since they are found on very small routed test sets and do not in any way suggest that they would be universally perfect in terms of the behaviour of the branches.

6. Design Comparison

The two design options meet the brief, however, they focus on varying architectural concepts. Design Choice 1 will be simpler and smaller: it will be trained on only three models, and the feature flow among chains is the same. Design Choice 2 is more loyal to hierarchical dependency in that subsequent models are only executed following previous forecasts, but requires more branch-specific models, and should be sensitive to sparse subsets

Dimension	Design Choice 1	Design Choice 2	Assessment view
Main idea	Combined targets	Prediction routing	Both are valid and distinct.
Models trained	3	8	DC2 is architecturally heavier.
Feature flow	Global and linear	Branching after prediction	DC2 better shows dependency.
Error propagation	Implicit in combined target	Explicit through routing	DC2 is more realistic for hierarchy.
Implementation complexity	Lower	Higher	DC1 is simpler to maintain.
Data sparsity sensitivity	Moderate	High	DC2 can skip weak branches safely.

To be defensible in submission, it is the simplest and more stable software architecture Design Choice 1, and the more demonstrative architectural Design Choice 2 since it brings out hierarchial dependency and propagation of errors because it does not keep out the hierarchical dependency or propagation of errors. As the brief values both architectural rationality and the architectural practice, it is correct to show both designs with this difference.

7. Conclusion

The codebase provided is in line with the CA requirements. It maintains separation of concerns, encapsulates input data and applies abstraction with a base model interface. It then makes both necessary multi-label design decisions of a common cleaned dataset and a validated TF-IDF feature space of form (206, 1519).

The last confirmed run indicates 206 retained records after preprocessing, 0.7619, 0.6061, and 0.7931 Design Choice 1 branch accuracy, and 0.7619 at Level 1, 0.7647 at Problem/Fault -> y3, 0.6667 at Suggestion -> y3, and 0.6429 at Suggestion + Payment -> y4. These values have been typed as they are found in the output of the execution.

On the whole, this reinvented report complies with the CA requirement of design sketches, architecture description, component- connector-data-element mapping and result discussion within a single coherent document.

Appendix A. Verified Execution Snapshot

- After preprocessing: 206 records remain
- y_chain_1 valid rows: 206 | Accuracy: 0.7619
- y_chain_2 valid rows: 171 | Accuracy: 0.6061
- y_chain_3 valid rows: 163 | Accuracy: 0.7931
- TF-IDF matrix shape: (206, 1519)
- DC2 trained models: 8

Appendix B. Source Code Access

The complete implementation of the email classifier, structured according to the principles of Separation of Concerns, Encapsulation, and Abstraction discussed in this report, is available online.

Link:
https://github.com/hasnat360/EEA_MultiLabel_Email_Classifier - https://github.com/hasnat360/EEA_MultiLabel_Email_Classifier